

Relazione tecnica per il corso di

Tecnologie del linguaggio naturale

Convertitore Lessicale Italiano $\rightarrow$ Yoda

Analisi sintattica con algoritmo CKY e trasformazione dell'ordine delle
parole

Franco Andrea

Iudice Samuele

Olivero Alessandro

Università degli Studi di Torino

Corso di Laurea Magistrale in Informatica

Anno Accademico 2025/2026

Sommario

L'obiettivo è l'implementazione di un sistema per la conversione automatica di frasi dall'italiano allo Yodish, variante sintattica ispirata al linguaggio di Yoda in Guerre Stellari. Sviluppato in Python, il software integra un parser CKY per l'analisi a costituenti e un modulo di trasformazione ricorsiva degli alberi di derivazione. Questa relazione analizza l'intero workflow computazionale, dalla modellazione della grammatica alla manipolazione sintattica, discutendo i risultati ottenuti e i limiti intrinseci dell'approccio rule-based.

Indice

1	Introduzione	3
2	Applicazioni teoriche	3
2.1	Grammatiche Context-Free	3
2.2	Forma Normale di Chomsky	3
2.3	L'Algoritmo CKY	4
3	Struttura Progettuale	4
3.1	Lexicon (<code>Lexicon.json</code>)	4
3.2	Le Regole Grammaticali (<code>regole_binarie.json</code>)	5
4	Implementazione	5
4.1	La Classe Nodo	5
4.2	L'Algoritmo CKY: Implementazione	6
4.3	La Trasformazione in Ordine Yoda	7
4.4	Linearizzazione e Post-processing	7
5	Interfaccia da Linea di Comando (CLI)	8
6	Esempio di Esecuzione	8
6.1	Albero di Derivazione Originale	8
6.2	Albero Trasformato in Ordine Yoda	9
7	Limiti e Possibili Sviluppi Futuri	9
7.1	Limiti del Sistema	9
7.2	Sviluppi Futuri	10
8	Conclusioni	10

1 Introduzione

Il progetto svolto presenta un sistema di conversione sintattica dall'italiano standard (SVX) all'ordine (XSV) tipico dello yodish. Sviluppato in Python, il software modella il linguaggio tramite grammatiche context-free (CFG) in Forma Normale di Chomsky (CNF).

Lo sviluppo progettuale si articola in tre fasi principali:

- **Parsing CKY**: analisi bottom-up della frase tramite un lessico e regole grammaticali definiti in formato JSON.
- **Trasformazione**: manipolazione ricorsiva dell'albero di derivazione per il riposizionamento dei costituenti sintattici.
- **Linearizzazione**: generazione della stringa finale a partire dalle foglie dell'albero trasformato.

L'approccio adottato garantisce una netta separazione tra logica algoritmica e base di conoscenza linguistica, consentendo maggiore flessibilità e portabilità del sistema.

2 Applicazioni teoriche

2.1 Grammatiche Context-Free

Una grammatica context-free è una quadrupla $G = (V, \Sigma, R, S)$ dove:

- V è un insieme finito di simboli non-terminali.
- Σ è un insieme finito di terminali.
- R è un insieme finito di regole di produzione della forma $A \rightarrow \alpha$.
- $S \in V$ è il simbolo iniziale detto assioma.

Una stringa $w \in \Sigma^*$ è *generata* da G se esiste una derivazione $S \Rightarrow^* w$. Il linguaggio $L(G)$ generato dalla grammatica è l'insieme di tutte le stringhe derivabili dall'assioma.

2.2 Forma Normale di Chomsky

Un algoritmo di parsing efficiente come CKY richiede che la grammatica sia in **Forma Normale di Chomsky** (CNF). Una CFG è in CNF se tutte le regole di produzione sono della forma:

$$A \rightarrow BC \quad \text{oppure} \quad A \rightarrow a$$

dove $A, B, C \in V$ e $a \in \Sigma$, ovvero, ogni regola produce esattamente due non-terminali oppure un singolo terminale.

2.3 L'Algoritmo CKY

L'algoritmo **CKY** (da Cocke, Kasami e Younger) è un algoritmo di parsing bottom-up per grammatiche in CNF, basato su programmazione dinamica. Dato l'input $w = w_1w_2 \cdots w_n$, l'algoritmo costruisce una tabella triangolare T di dimensione $n \times n$, dove la cella $T[i][j]$ contiene l'insieme dei non-terminali che possono derivare la sottostringa $w_iw_{i+1} \cdots w_j$.

Algoritmo:

1. **Inizializzazione:** per ogni i da 1 a n , inserire in $T[i][i]$ tutti i non-terminali A tali che $A \rightarrow w_i$ è una regola lessicale.
2. **Riempimento:** per ogni lunghezza di span ℓ da 2 a n , per ogni inizio i e fine $j = i + \ell - 1$, per ogni punto di taglio k da i a $j - 1$: se $B \in T[i][k]$ e $C \in T[k+1][j]$ e $A \rightarrow BC \in R$, allora inserire A in $T[i][j]$.
3. **Accettazione:** la frase è riconosciuta se e solo se $S \in T[1][n]$.

L'efficienza dell'algoritmo deriva dalla tecnica della programmazione "dinamica", che evita di ricalcolare più volte gli stessi sotto-alberi sintattici, risolvendo il problema del parsing in tempo polinomiale invece che esponenziale.

3 Struttura Progettuale

Il progetto è organizzato in tre componenti principali:

File	Tipo	Descrizione
<code>convertitore_yoda.py</code>	Codice Python	Modulo principale: parsing, trasformazione, CLI
<code>Lexicon.json</code>	Dati JSON	Lessico italiano con etichette POS
<code>regole_binarie.json</code>	Dati JSON	Regole binarie della grammatica in CNF

3.1 Lexicon (Lexicon.json)

Lexicon è un dizionario in file JSON che mappa ogni parola italiana a una lista di categorie grammaticali (POS tag). Ad esempio:

Le categorie utilizzate nella grammatica sono: **Det** (determinante), **N** (nome), **NP** (nome proprio / sintagma nominale pre-formato), **V** (verbo), **Adj** (aggettivo), **Adv** (avverbio), **Prep** (preposizione), **Num** (numerale).

3.2 Le Regole Grammaticali (`regole_binarie.json`)

Il file contiene le regole binarie della grammatica in CNF, dove la chiave è la coppia di figli e il valore è la lista dei possibili genitori. La scelta di invertire la direzione rispetto alla notazione tradizionale (dai figli al genitore) è una ottimizzazione utile all'algoritmo CKY: durante il parsing si cerca quali genitori sono compatibili con una coppia di figli già riconosciuti, non il contrario.

```
1 {  
2   "Det , N":    ["NP"] ,  
3   "NP , VP":    ["S"] ,  
4   "V , NP":     ["VP" , "VP1"] ,  
5   "VP1 , Adv":  ["VP"] ,  
6   "Prep , NP":  ["PP"] ,  
7   "V , PP":     ["VP"] ,  
8   "V , Adj":    ["VP"] ,  
9   "Num , N":    ["NP"]  
10 }
```

Listing 1: Esempio di regole binarie (formato chiave-valore invertito)

La funzione `carica_dizionari()` in fase di inizializzazione converte le chiavi stringa (es. "Det,N") nelle tuple Python richieste dall'algoritmo (es. ('Det', 'N')).

4 Implementazione

4.1 La Classe Nodo

L'albero di derivazione è rappresentato tramite la classe `Nodo`, che implementa un nodo di albero n-ario con tre attributi:

- **label**: il simbolo grammaticale del nodo (es. 'NP', 'VP') oppure la parola stessa nelle foglie.
- **children**: lista di nodi figli.
- **word**: la parola (solo per le foglie lessicali).

I nodi foglia (terminali) sono distinti da quelli interni tramite il metodo `is_leaf()`, che verifica se la lista `children` è vuota.

La struttura è volutamente semplice per facilitare la visita ricorsiva sia nella stampa (`pretty_print`) che nella trasformazione Yoda.

```

1 class Nodo:
2     def __init__(self, label, children=None, word=None):
3         self.label      = label
4         self.children   = children or []
5         self.word       = word
6
7     def is_leaf(self):
8         return len(self.children) == 0

```

Listing 2: Classe Nodo – costruttore e metodi principali

Il metodo `pretty_print()` produce una rappresentazione visuale ad albero nel terminale, con connettori grafici ASCII (`|`, `├`, `└`) analoga a quella del comando Unix `tree`.

4.2 L'Algoritmo CKY: Implementazione

La funzione `cky_parse(sentence)` riceve una lista di parole tokenizzate e restituisce il nodo radice dell'albero di derivazione (etichettato 'S') oppure `None` se la frase non è riconosciuta dalla grammatica.

L'implementazione usa una tabella tridimensionale `table[i][j]` dove ogni cella è un dizionario `{simbolo: Nodo}`. Questa scelta — memorizzare direttamente il nodo invece del semplice booleano — consente di ricostruire l'albero durante il parsing senza una fase separata di *backtracking*.

```

1 for span in range(2, n + 1):
2     for i in range(n - span + 1):
3         j = i + span - 1
4         for k in range(i, j):
5             for (B, C), parents in binary_rules.items():
6                 if B in table[i][k] and C in table[k+1][j]:
7                     for A in parents:
8                         if A not in table[i][j]:
9                             node_B = table[i][k][B]
10                            node_C = table[k+1][j][C]
11                            table[i][j][A] = Nodo(
12                                label=A,
13                                children=[node_B, node_C]
14                            )

```

Listing 3: Nucleo dell'algoritmo CKY con ricostruzione dell'albero

Si noti che, in caso di ambiguità grammaticale, l'algoritmo mantiene solo il *primo* albero trovato per ogni non-terminale in ogni cella, corrispondente alla prima regola applicabile nell'ordine di iterazione del dizionario. Una grammatica ambigua produce risultati deterministici ma dipendenti dall'ordinamento delle regole.

4.3 La Trasformazione in Ordine Yoda

Il modulo più originale del progetto è la funzione `trasforma_in_yoda()`, che visita l'albero in post-ordine (prima i figli, poi il nodo corrente) e riorganizza i sottoalberi laddove trova la struttura sintattica canonica italiana.

La regola principale è: ogni nodo S con struttura $S \rightarrow NP VP$ viene trasformato in $S_yoda \rightarrow Comp NP V$, dove **Comp** è il complemento contenuto nel VP. In altre parole, il complemento viene spostato in posizione iniziale di frase.

Sono gestiti tre casi distinti:

Caso 1: Frase transitiva semplice $VP \rightarrow V X$ con $X \in \{NP, PP, Adj\}$

$$S \rightarrow NP (VP \rightarrow V Comp) \implies S_yoda \rightarrow Comp NP V$$

Caso 2: Frase con avverbio $VP \rightarrow VP1 Adv$ con $VP1 \rightarrow V NP$

$$S \rightarrow NP (VP \rightarrow (VP1 \rightarrow V NP) Adv) \implies S_yoda \rightarrow NP Sogg V Adv$$

Caso 3: Frase intransitiva $VP \rightarrow V$ (solo verbo)

$$S \rightarrow NP (VP \rightarrow V) \implies S_yoda \rightarrow NP V$$

(nessuna trasposizione, l'ordine resta invariato poiché non c'è complemento da anteporre)

```

1 if child0.label == 'V' and child1.label in ('NP', 'PP', 'Adj'):
2     verbo = child0
3     compl = child1
4     nodo.label = 'S_yoda'
5     nodo.children = [compl, sogg, verbo]
```

Listing 4: Trasformazione Caso 1 – VP con complemento diretto

4.4 Linearizzazione e Post-processing

La funzione `raccogli_foglie()` effettua una visita DFS (depth-first, in-order) sull'albero trasformato, raccogliendo le parole nelle foglie da sinistra a destra. La funzione `converti_in_yoda()` applica infine un post-processing minimo:

- Le parole vengono rese minuscole, salvo i *nomi propri*, che vengono identificati come parole che appaiono nel lessico con entrambi i tag N e NP e iniziano con la maiuscola.
- La prima parola della frase Yoda viene capitalizzata.

5 Interfaccia da Linea di Comando (CLI)

Il modulo è utilizzabile direttamente da terminale grazie al modulo `argparse` della libreria standard Python. Sono supportate due modalità d'uso:

Frase singola:

```
1 python yoda.py "Mario mangia il gatto"
```

Modalità test:

```
1 python yoda.py --test
```

La modalità `-test` esegue una suite di frasi predefinite confrontando l'output ottenuto con quello atteso. Ogni test riporta CORRETTO o ERRORE con il dettaglio dell'output ottenuto vs. atteso.

Le tre frasi di test incluse nel codice sono:

Tabella 1: Suite di test del modulo `-test`

Input italiano	Output Yoda atteso
Tu hai amici lì	Amici tu hai lì
Tu avrai novecento anni di età	Novecento anni di età tu avrai
Noi siamo illuminati	Illuminati noi siamo

L'output a schermo include, per ogni frase: la frase originale, l'albero di derivazione iniziale, l'albero dopo la trasformazione Yoda, e la frase finale. Questo livello di verbosità è intenzionale a scopo didattico.

6 Esempio di Esecuzione

Per illustrare concretamente il funzionamento del sistema, si considera la frase di input *"Tu hai amici lì"*.

6.1 Albero di Derivazione Originale

La frase viene tokenizzata in [Tu, hai, amici, lì]. Il parser CKY riconosce la struttura seguente:

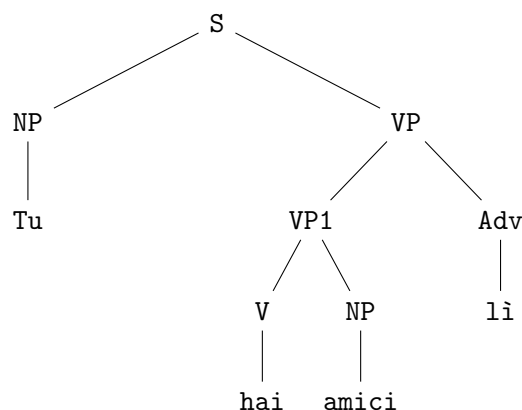


Figura 1: Albero di derivazione della frase “Tu hai amici lì”

6.2 Albero Trasformato in Ordine Yoda

La funzione `trasforma_in_yoda()` identifica la struttura $S \rightarrow NP (VP \rightarrow (VP1 \rightarrow V NP) Adv)$ (Caso 2) e riorganizza le costituenti nell’ordine *Comp Sogg V Adv*:

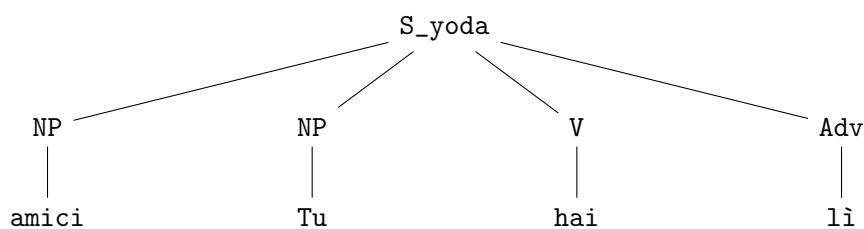


Figura 2: Albero trasformato in ordine Yoda: “Amici tu hai lì”

La linearizzazione delle foglie produce la frase finale:

Input: *Tu hai amici lì*

Output IT-YO: *Amici tu hai lì*

7 Limiti e Possibili Sviluppi Futuri

7.1 Limiti del Sistema

Copertura e Scalabilità. Il lessico e la grammatica sono limitati a costruzioni semplici (SVO). L’estensione manuale dei file JSON non è scalabile per il linguaggio naturale arbitrario e non sono gestite strutture complesse (subordinate, passive, clitici).

Gestione dell’Ambiguità. Il sistema restituisce il primo albero trovato senza segnalare eventuali ambiguità strutturali né permettere l’esplorazione di parsing alternativi.

Analisi Morfologica. Manca un modulo di lemmatizzazione: il lessico si basa su forme di superficie, richiedendo l'inserimento manuale di ogni flessione verbale o nominale.

Trasformazione Superficiale. La "Yodificazione" avviene efficacemente solo al livello radice (S); eventuali subordinate annidate non vengono processate correttamente.

7.2 Sviluppi Futuri

- **Integrazione Risorse Esterne:** Uso di spaCy o WordNet per la copertura lessicale e il PoS tagging automatico.
- **Arricchimento Grammaticale:** Supporto a frasi relative, coordinate e perifrasi verbali complesse.
- **Parser Probabilistico:** Implementazione di **PCFG** con algoritmo CKY probabilistico per una gestione pesata dell'ambiguità.
- **Interfaccia Utente:** Sviluppo di una GUI o web app (Flask) per rendere il tool accessibile.
- **Trasformazioni Multiple:** Implementazione di altri stili sintattici (es. topicalizzazione o dislocazione).

8 Conclusioni

Il progetto IT-YO ha consentito di mettere in pratica i concetti fondamentali di analisi sintattica formale: progettazione di una grammatica context-free in Forma Normale di Chomsky, implementazione dell'algoritmo CKY con ricostruzione dell'albero di derivazione, e manipolazione strutturale dell'albero per ottenere una trasformazione dell'ordine delle parole.

Il sistema è funzionante per un sottoinsieme controllato di frasi italiane e produce output corretti sui casi di test previsti. La separazione tra grammatica (dati JSON) e logica di parsing (codice Python) rende il sistema facilmente estendibile: per coprire nuove costruzioni sintattiche è sufficiente aggiungere voci al lessico e regole al file JSON, senza modificare l'algoritmo.

Dal punto di vista didattico, il progetto illustra concretamente come la teoria delle grammatiche formali si traduca in un sistema computazionale funzionante, e come la manipolazione degli alberi di derivazione sia uno strumento potente per operare trasformazioni linguistiche strutturate.

Tra i possibili sviluppi futuri, l'integrazione con un PoS tagger automatico e l'estensione della grammatica a costruzioni più complesse rappresentano le direzioni più promettenti per avvicinare il sistema a un uso su frasi naturali non vincolate.